



Universidade Estadual de Campinas
Faculdade de Engenharia Civil
IC907-Inteligência Artificial Aplicada a Problemas
de Engenharia

Projeto 2 - Estimação de propriedades materiais via equação diferencial

Gabriel Silva 235038
Luis Felipe Oliveira Santos 299675
Yan de Miranda Zimmermann Lobo 195649

Campinas/SP
2025

Introdução

Redes neurais artificiais (ANN do inglês *Artificial Neural Networks*) são um tipo de aprendizagem de máquina, isto é, de maneira simplificada, um método para qual seja possível que máquinas por meio da exposição a diferentes dados aprendam e realizem tarefas de maneira autônoma. A concepção teórica de uma rede neural é inspirada no funcionamento do neurônio humano, ou seja, para que ocorra o processamento de um dado de entrada há vários neurônios, ativações e interconectividades que resultam em uma informação processada [1, 2]. Essa interconectividade entre neurônios é um dos grandes diferenciais das ANNs, pois permite, por exemplo, o aprendizado considerando possíveis interdependência entre dados. Uma ANN é formada, simplificadamente, por diferentes camadas, sendo uma primeira camada relativa a dados de entrada e a última aos resultados relativos aos dados de entrada. As camadas intermediárias são chamadas de camadas densas (encontrado na literatura em inglês como *Dense Layers*), elas são compostas por diversos neurônios que são interconectados entre essas camadas, isto é, cada neurônio de uma camada densa é conectado com cada neurônio da próxima camada densa. O aprendizado da ANN é realizado por meio do ajuste de pesos e constantes (*weights* e *biases*), sendo esses pesos ajustados para cada conexão entre neurônios e as constantes para cada neurônio [3, 4]. Nesse sentido, as ANNs podem ser interpretadas como uma sequência de transformações lineares, cujo resultado passa por uma função de ativação, o que permite as ANNs generalizarem uma grande variedade de problemas. Entretanto, as redes neurais apresentam limitações quando se tem como objetivo incorporar restrições físicas ao problema, além disso, na maioria das vezes, o custo da aquisição de dados para treinamento das redes neurais é muito alto, o que leva a cenários onde a rede neural carece de robustez e não oferece nenhuma garantia de convergência. Deste modo, as PINNs (do inglês *Physics-Informed Neural Networks*) aparecem como uma solução para possibilitar com que a rede neural tenha conhecimento acerca do problema físico, por meio da modificação da função custo do treinamento, ou seja, termos de penalização são adicionados a rede quando as soluções não respeitam a física informada [5].

Metodologia

O objetivo principal do presente trabalho é estimar as propriedades dos materiais via equação diferencial. Para isso, inicialmente foi desenvolvida uma rede neural utilizando a biblioteca PyTorch. A fim de uma validação inicial do código, diversos testes foram gerados: diferentes valores de tamanho de amostra, níveis de ruído, valores de pesos das funções custo e tamanho da rede; para a equação diferencial estudada, que será apresentada a seguir.

A equação diferencial utilizada para treino da rede neural utilizando PINNs foi a equação diferencial de Darcy para fluxo transiente em 1 dimensão. A equação diferencial é mostrada abaixo, onde ϕ é a porosidade do meio (adimensional), c é a compressibilidade total – fluido mais matriz – (Pa^{-1}), k é a permeabilidade (m^2) e μ é a viscosidade do fluido (Pa.s).

$$\frac{\partial^2 P}{\partial x^2} = \frac{\phi \mu c}{k} \frac{\partial P}{\partial t}$$

Para geração dos pontos de treino foi utilizada a seguinte equação analítica, com 100 termos no somatório infinito. A equação analítica foi obtida de notas de aula encontradas em [6].

$$P(x,t) = P_L + (P_R - P_L) \left[\frac{x}{L} + \frac{2}{\pi} \sum_{n=1}^{infinit} \frac{1}{n} \exp\left(-\frac{n^2 \pi^2}{L^2} \frac{k}{\phi \mu c} t\right) \sin\left(n \pi \frac{x}{L}\right) \right]$$

Para testes da PINN foram utilizados os parâmetros fictícios mostrados na Tabela 1.

Tabela 1: Parâmetros fictícios utilizados para caracterização do problema.

Parâmetro	Valor	Unidade
Posição inicial	0	m
Posição final	1	M
Pressão esquerda	1	Pa
Pressão direita	0	Pa
Permeabilidade (k)	1e-12	m²
Viscosidade dinâmica (μ)	1e-3	Pa.s
Porosidade (φ)	0.2	-
Compressibilidade total (c)	1e-9	1/Pa

Tempo inicial	0.0001	s
Tempo final	0.1	s

A função analítica com os parâmetros utilizados resulta na superfície mostrada na Figura 1.

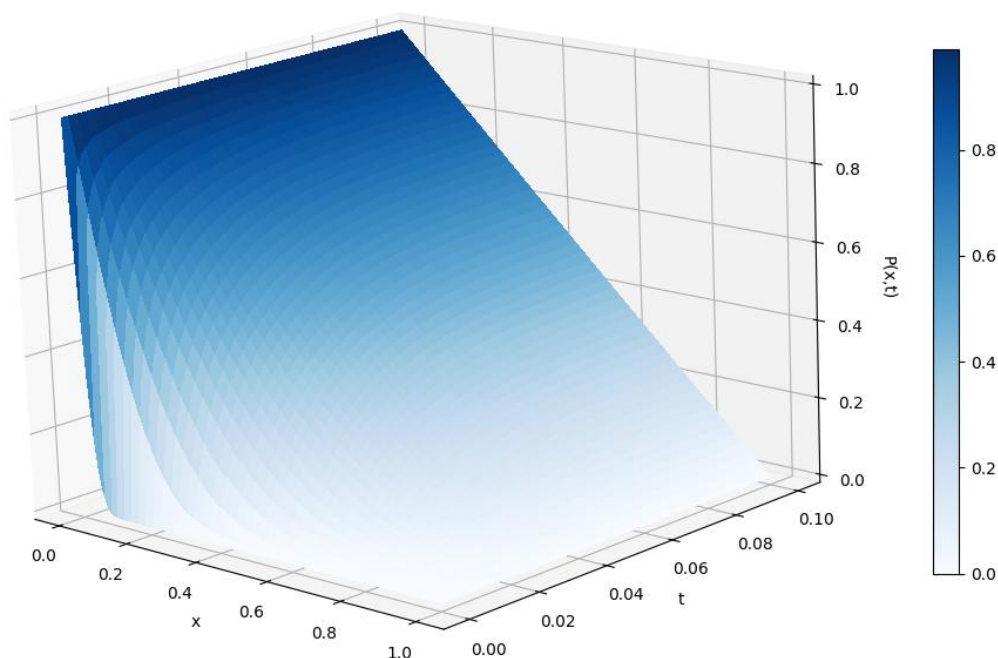


Figura 1: Função analítica avaliada para os parâmetros utilizados.

Após a realização de diversos experimentos, conforme explicitado anteriormente, e baseado nas observações acerca da influência de cada hiperparâmetro, foi estipulado o melhor cenário de hiperparâmetro. Com isso, foi implementado uma rede neural para estimar os valores de coeficientes da equação diferencial. Foi escolhida a porosidade para apresentação dos resultados.

Sobre o código da ANN

A arquitetura do código da rede neural para aplicação de PINNs é apresentada a seguir. Ele é organizado, basicamente, a partir da classe PINN, mostrada na Figura 2. A classe recebe os parâmetros necessários para treinamento de uma rede neural e, em complemento, três parâmetros que possibilitam a inclusão de funções peso relativas a condições de contorno, condições iniciais, equações diferenciais, dentre outras; a inclusão

de classes que definem uma equação diferencial e a identificação de coeficientes a serem estimados pela rede.

```
class PINN(nn.Module, BasicData):
    """
    Feed-forward neural network with configurable hidden layers.

    Parameters
    - input_dim (int): number of input features (e.g., 1 for position x).
    - hidden_layers (iterable[int]): sizes of hidden layers, e.g. [32, 16].
    - output_dim (int): number of outputs (e.g., 1 for deflection v(x)).
    - activation (nn.Module): activation class used between hidden layers (default: nn.ReLU).
    - epochs (int): number of training epochs.
    - loss (callable): loss function (default: nn.MSELoss()).
    - lr (float): optimizer learning rate.

    Notes
    - Uses ReLU (or provided activation) after each hidden layer and a linear output layer.
    - The forward() implementation accepts 1D tensors (unsqueezes to shape (N, input_dim)).
    """
    def __init__(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr=float=0.01, ...):
        ...
    def forward(self, x): ...
    def predict(self, x) -> torch.Tensor: ...
    def np_to_th(self, x): ...
    def train_nn(self, x_train:np.ndarray=None, y_train:np.ndarray=None): ...
    def plot_loss(self): ...
    def plot_prediction(self, numpoints:int=100): ...
```

Figura 2: Classe PINN.

Os parâmetros descritos anteriormente (LossPINNVec, problem, parameter_discovery), são destacados na Figura 3.

```
def __init__(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr=float=0.01,
             LossPINNVec:list[LossPINN]=[], Problem:DarcyTransientFlow=None, parameter_discovery:list[str]=None):
```

Figura 3: Parâmetros principais da classe PINN.

O LossPINNVec é uma lista que recebe parâmetros do tipo LossPINN, que são métodos estruturados conforme mostra a Figura 4, isto é, eles recebem dois parâmetros (o modelo e o número de pontos) e retornam o valor do custo.

```

    """ Boundary condition  $P(0, t) = PL$ 
    def loss_bc_left(model:PINN, numpoints:int=100):
        x0 = np.array([[0, t] for t in np.linspace(model.Problem.startTime, model.Problem.endTime, numpoints)])
        x0_torch = model.np_to_th(x0).requires_grad_(True)
        P0_pred = model(x0_torch)

        loss_bc_left = (P0_pred - model.Problem.PLeft)**2
        loss_bc_left = torch.mean(loss_bc_left)

        return loss_bc_left

    """ Boundary condition  $P(L, t) = PR$ 
    def loss_bc_right(model:PINN, numpoints:int=100): ...

    """ Physics loss (PDE residual)
    def physics_loss(model:PINN): ...

    """ Loss initial condition
    def loss_ic(model: PINN, numpoints=100): ...
```

Figura 4: Estrutura dos métodos de função custo.

O parâmetro `problem` é do tipo `DarcyTransientFlow`, que é responsável pelo cálculo da função, sendo mostrada na Figura 5.

```
class DarcyTransientFlow(BasicData):
    def __init__(self, L:float, PLeft:float, PRight:float, k:float, mu:float, phi:float, ct:float, startTime:float=0.001, endTime:float=1.0):
        super().__init__()
        self.L = L          #* Length of the domain (m)
        self.PLeft = PLeft  #* Pressure at the left boundary (Pa)
        self.PRight = PRight #* Pressure at the right boundary (Pa)
        self.k = k          #* Permeability (m^2)
        self.mu = mu        #* Dynamic viscosity (Pa.s)
        self.phi = phi       #* Porosity (-)
        self.ct = ct        #* Total compressibility (1/Pa)

        self.startTime = startTime
        self.endTime = endTime

        self.DeactivateAttr()

    def InfiniteSum(self, x:float, t:float, NInf:int=100) -> float: ...
    def AnalyticalSolution(self, x:float, t:float, NInf:int=100) -> float: ...
    def Plot(self): ...
```

Figura 5: Estrutura da classe `DarcyTransientFlow`.

O parâmetro `parameter_discovery` é uma lista de string que pode receber os seguintes valores: `["phi"]`, `["k"]`, `["mu"]`, `["ct"]`. Esse parâmetro é responsável por informar a rede se há e quais os coeficientes que devem ser estimados, como pode ser observado na Figura 6.

```
self.parameter_discovery: list[str] = parameter_discovery
self.unknown_parameters: list = list()
if self.parameter_discovery != None:
    for parameter in self.parameter_discovery:
        if parameter == "phi":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.phi, dtype=torch.float32), requires_grad=True))
        if parameter == "k":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.k, dtype=torch.float32), requires_grad=True))
        if parameter == "mu":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.mu, dtype=torch.float32), requires_grad=True))
        if parameter == "ct":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.ct, dtype=torch.float32), requires_grad=True))
```

Figura 6: Especificação dos coeficientes a serem estimados.

Uma outra classe importante para as análises, que não é diretamente relacionada a classe PINN, é a classe `DataSet`, Figura 7.

```
class DataSet(BasicData):
    def __init__(self, x:np.ndarray, y:np.ndarray):
        super().__init__()
        self.x = x
        self.y = y

    def split(self, split_rate:float = 0.75): ...
    def add_noise(self, randomization_factor:float=1.): ...
```

Figura 7: Classe `DataSet`.

Na classe DataSet é realizada a geração de ruídos para os pontos de treinamento, como pode ser observado na Figura 8. A geração de ruído é feita por meio de um fator de ruído multiplicado pela geração aleatória de valores entre -1 e 1.

```
def add_noise(self, randomization_factor:float=1.):
    """
    The random values is by default between -1 and 1.\n
    Use the randomization_factor to increase or decrease it.
    """
    ypts = copy.deepcopy(self.y)
    num_points: int = len(ypts)

    noise_points = (np.random.normal(-1., 1., num_points) * randomization_factor)
    y_randomized = ypts + noise_points.reshape(num_points, 1)

    self.y = copy.deepcopy(y_randomized)

    return
```

Figura 8: Método de geração de ruídos.

Experimentos

Para referência nas análises mostradas a seguir, foi utilizada uma rede neural base com número de amostras, hiperparâmetros e pesos das funções custo mostradas na Tabela 2. Os resultados foram gerados com *seed* igual a zero (*numpy* e *torch*) para garantir a reprodutibilidade dos resultados.

Tabela 2: Dados de referência para estudo da ANN

Número de amostras	1000
Fator de ruído	0.1
Taxa de aprendizado	0.01
Épocas	5000
Camadas ocultas	[50, 50, 50, 50]
Peso – condições de contorno	100
Peso – equação diferencial	1
Peso – condição inicial	50
Função de ativação	Tangente Hiperbólica

Os resultados da rede neural de referência são mostrados na Figura 9, sendo, respectivamente, as curvas de custo por época e valor da função por posição para diferentes tempos.

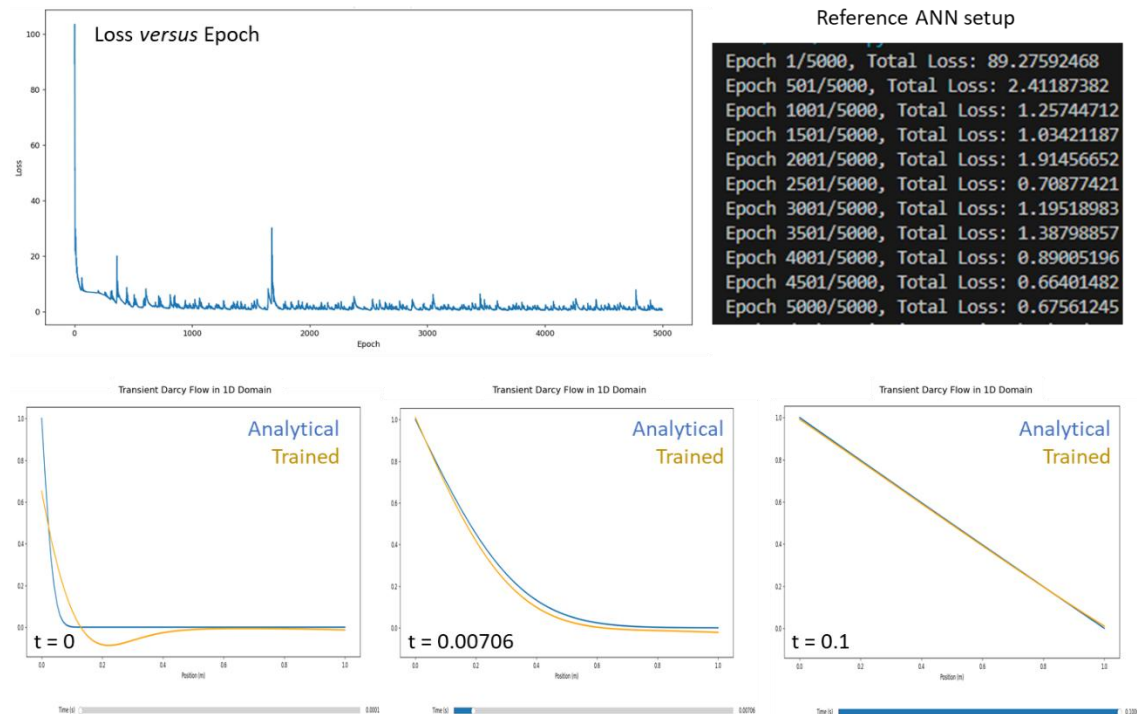


Figura 9: Resultados da ANN de referência.

Nas subseções a seguir são apresentados os resultados específicos da influência da quantidade de amostras, do nível de ruído, dos valores de peso das funções e do tamanho da rede e da função de ativação no resultado obtido para treinamento da equação diferencial de Darcy para fluxo transiente em 1 dimensão.

Influência da quantidade de amostras

Para estudo da influência do tamanho da amostra nos resultados obtidos, foram realizados treinamentos para 5000 e 10000 amostras de teste. Deste modo, a rede de treinamento de referência foi alterada primeiramente para 5000 amostras e depois para 10000 amostras, sendo os resultados mostrados na Figura 10 e Figura 11, respectivamente.

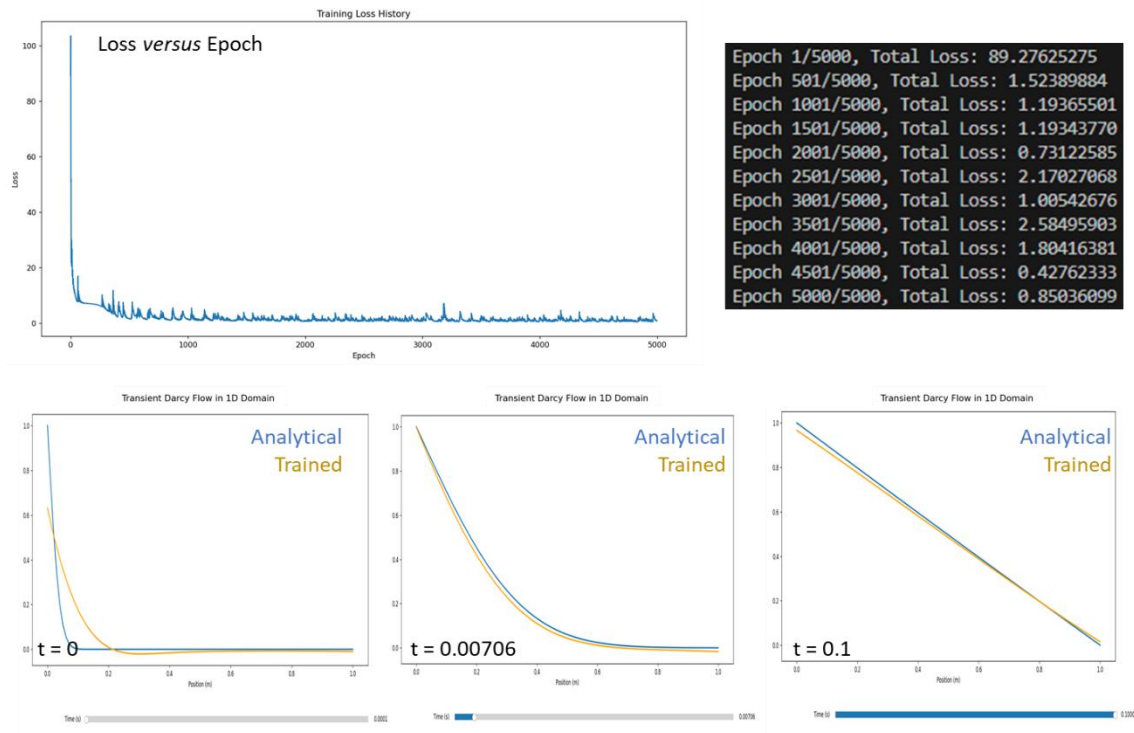


Figura 10: Resultados da ANN com 5000 amostras.

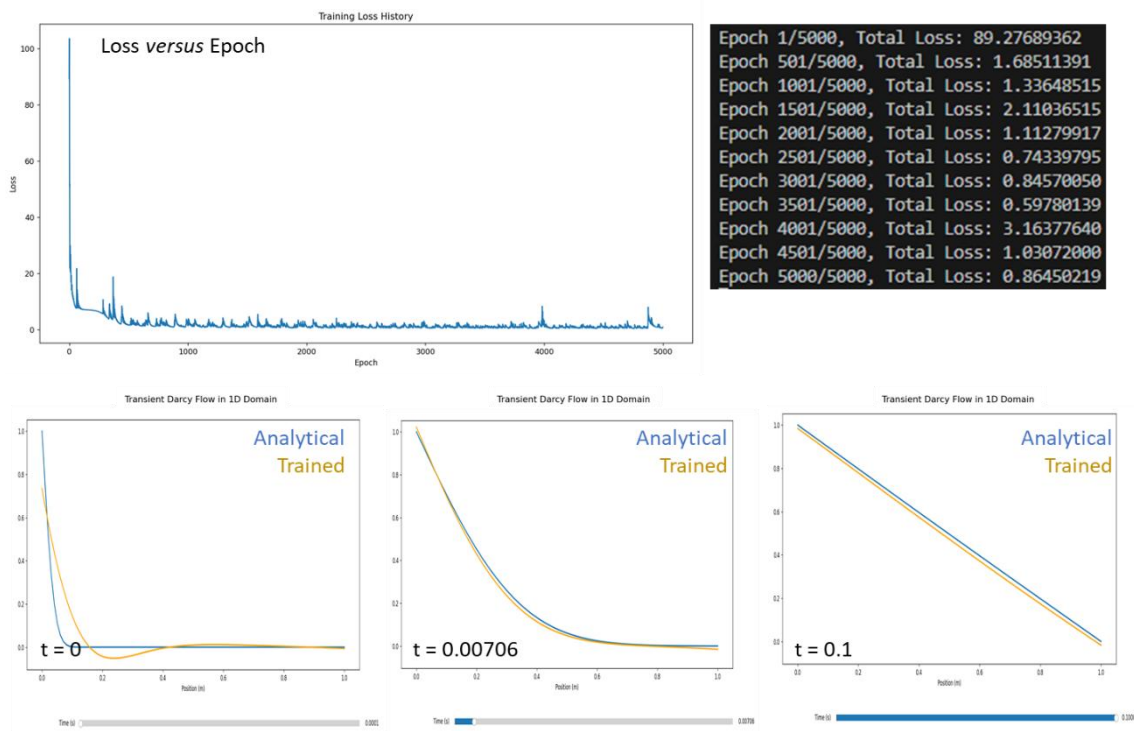


Figura 11: Resultados da ANN com 10000 amostras.

Influência do nível de ruído

Para estudo da influência do ruído nos resultados obtidos, o mesmo procedimento foi aplicado. Deste modo, três níveis de ruído foram treinados, sendo os fatores treinados iguais a 0, 0.3 e 0.5, mostrados na Figura 12, Figura 13 e Figura 14, respectivamente.

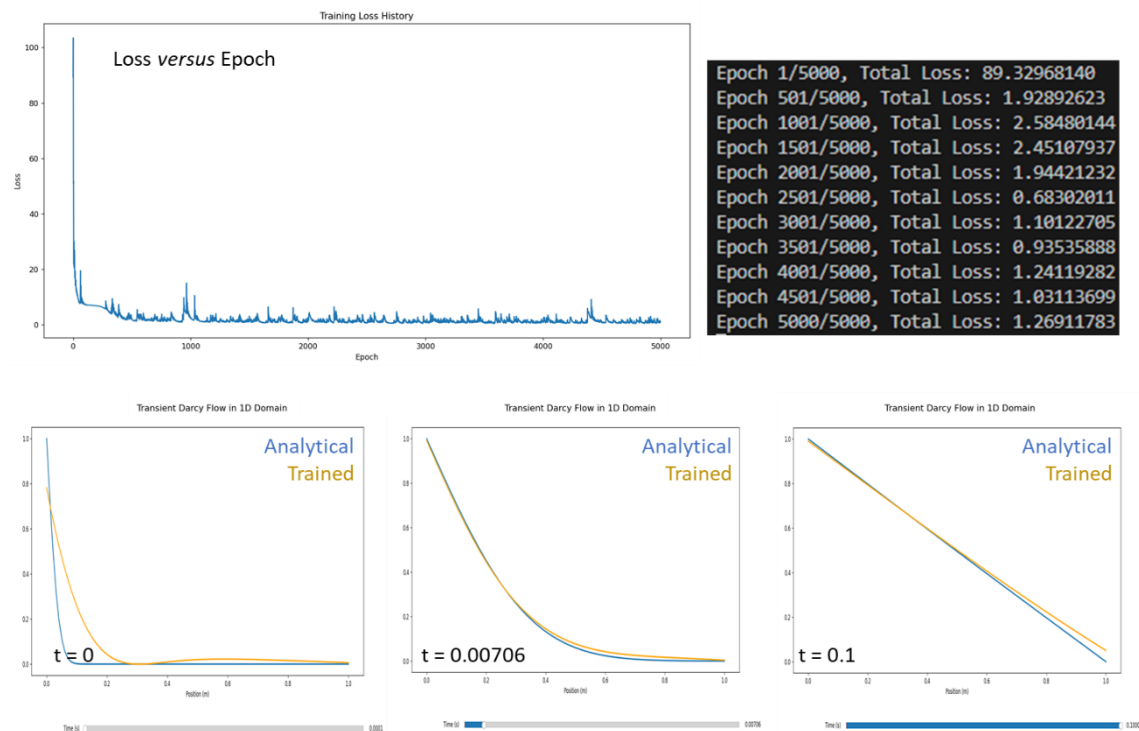


Figura 12: Resultados da ANN com ruído igual a zero.

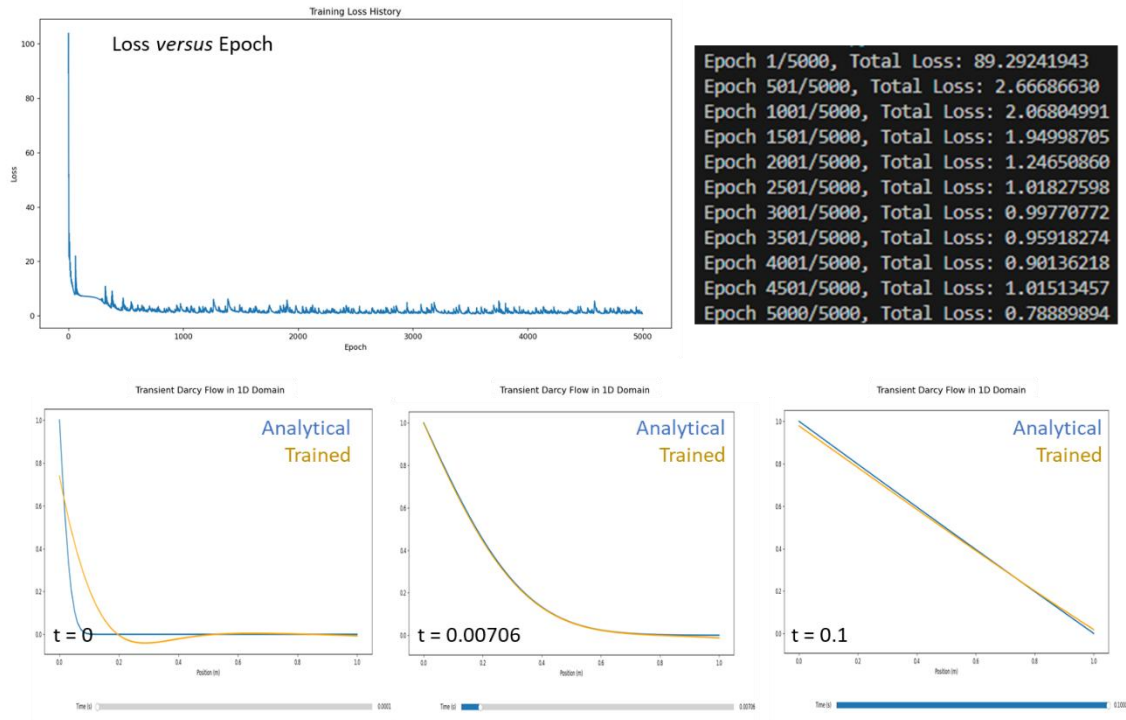


Figura 13: Resultados da ANN com ruído igual a 0.3.

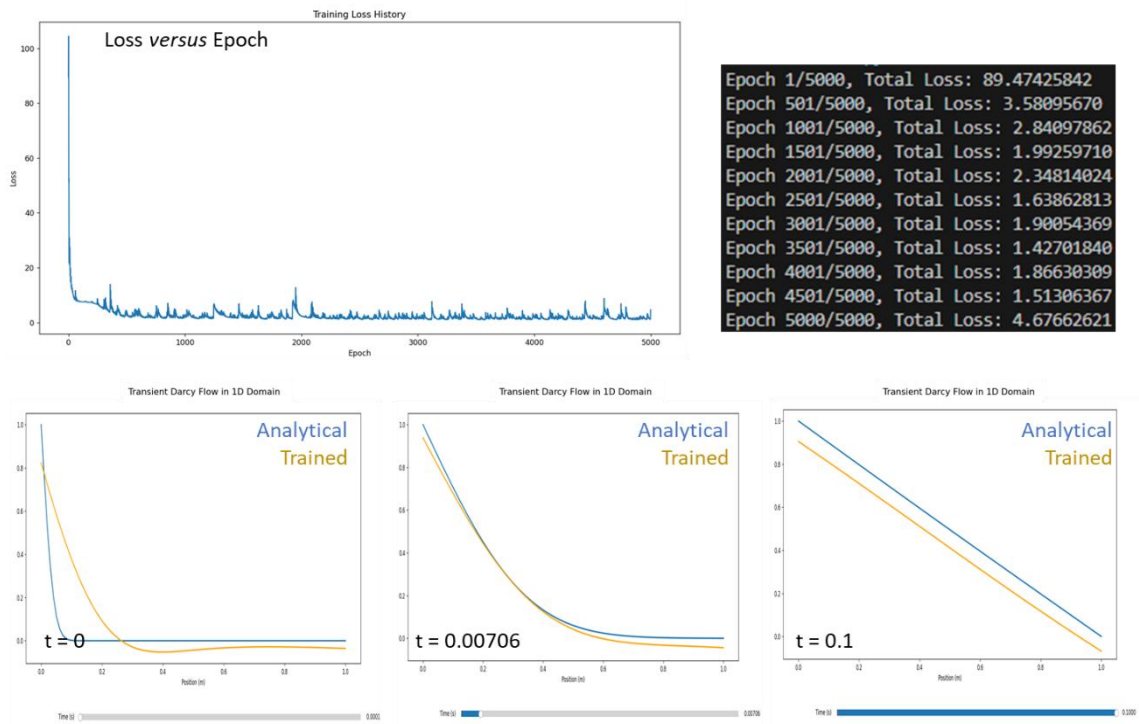


Figura 14: Resultados da ANN com ruído igual a 0.5.

Influência dos pesos das funções custo

Para verificar a influência dos pesos das funções custo adotou-se como critério: maiores valores de peso para as condições de contorno (BCs), seguida pela condição inicial (IC) e, por fim, a equação diferencial (ED). Deste modo, três casos foram testados, sendo:

- Caso 1: BCs = 1000 - IC = 50 - ED = 1 (Figura 15);
- Caso 2: BCs = 1000 - IC = 500 - ED = 1 (Figura 16);
- Caso 3: BCs = 1000 - IC = 500 - ED = 10 (Figura 17).

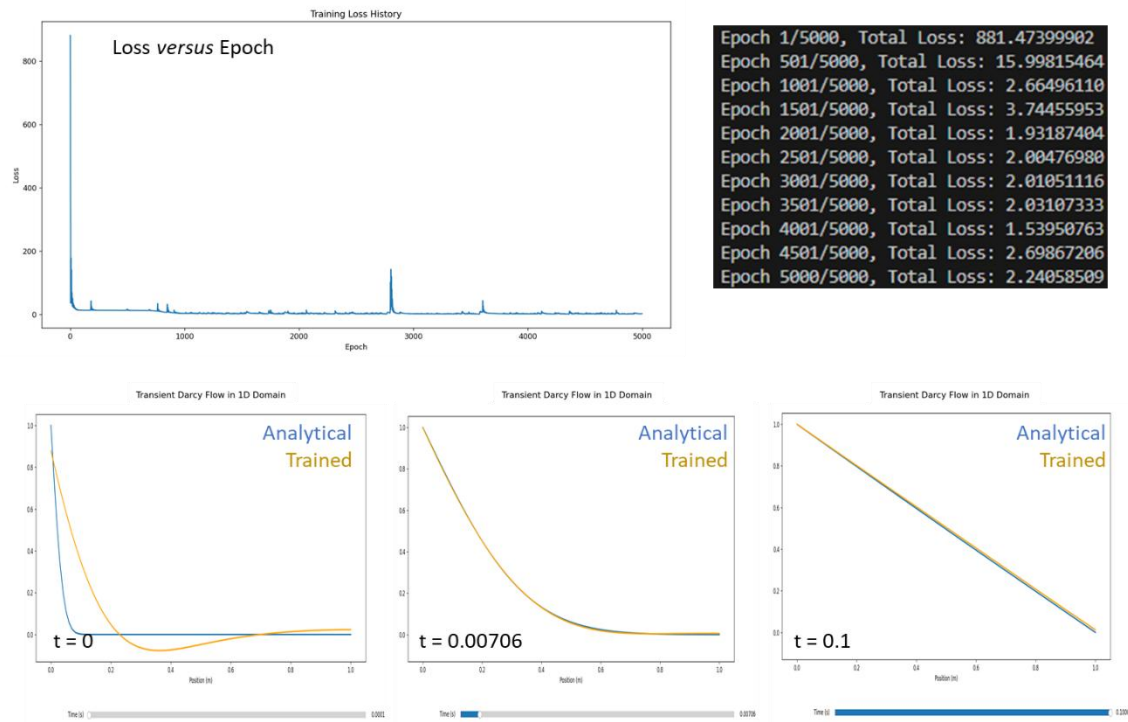


Figura 15: Resultados da ANN com pesos do caso 1.

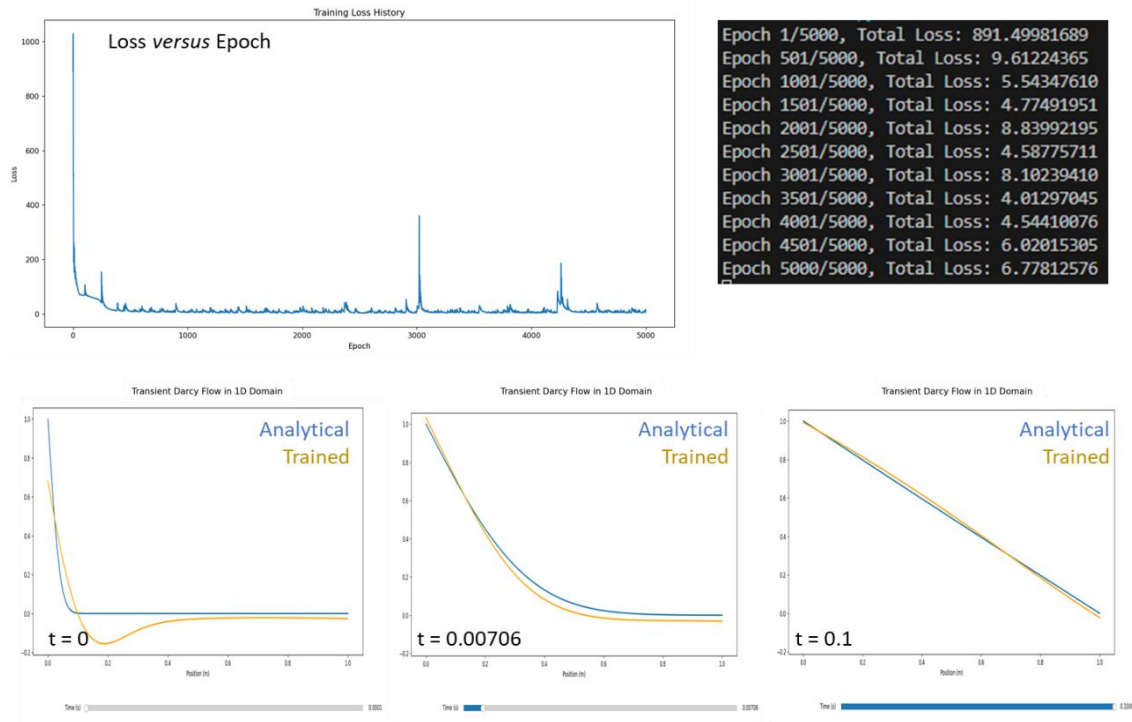


Figura 16: Resultados da ANN com pesos do caso 2.

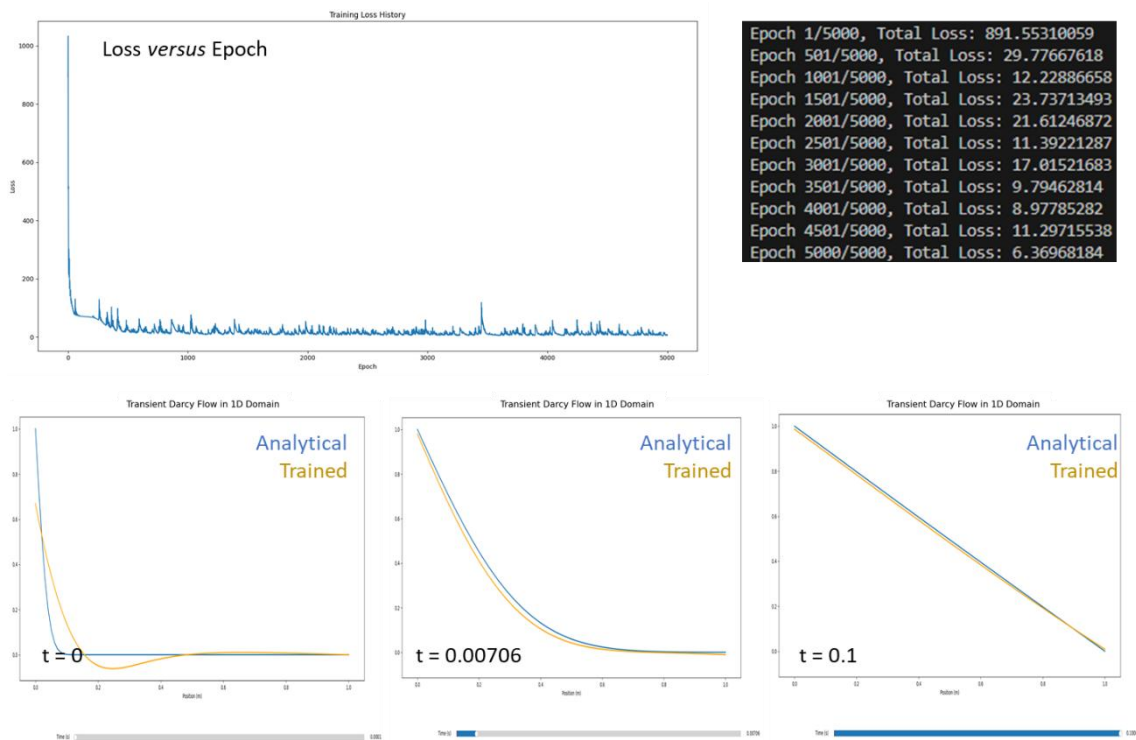


Figura 17: Resultados da ANN com pesos do caso 3.

Influência do tamanho da rede

Para testar a influência do tamanho da rede, as seguintes configurações de camadas ocultas foram testadas:

- Caso 1: [50, 50] (Figura 18);
- Caso 2: [50, 50, 50, 50, 50, 50] (Figura 19);
- Caso 3: [100, 100, 100, 100] (Figura 20);
- Caso 4: [100, 50, 25, 5] (Figura 21).

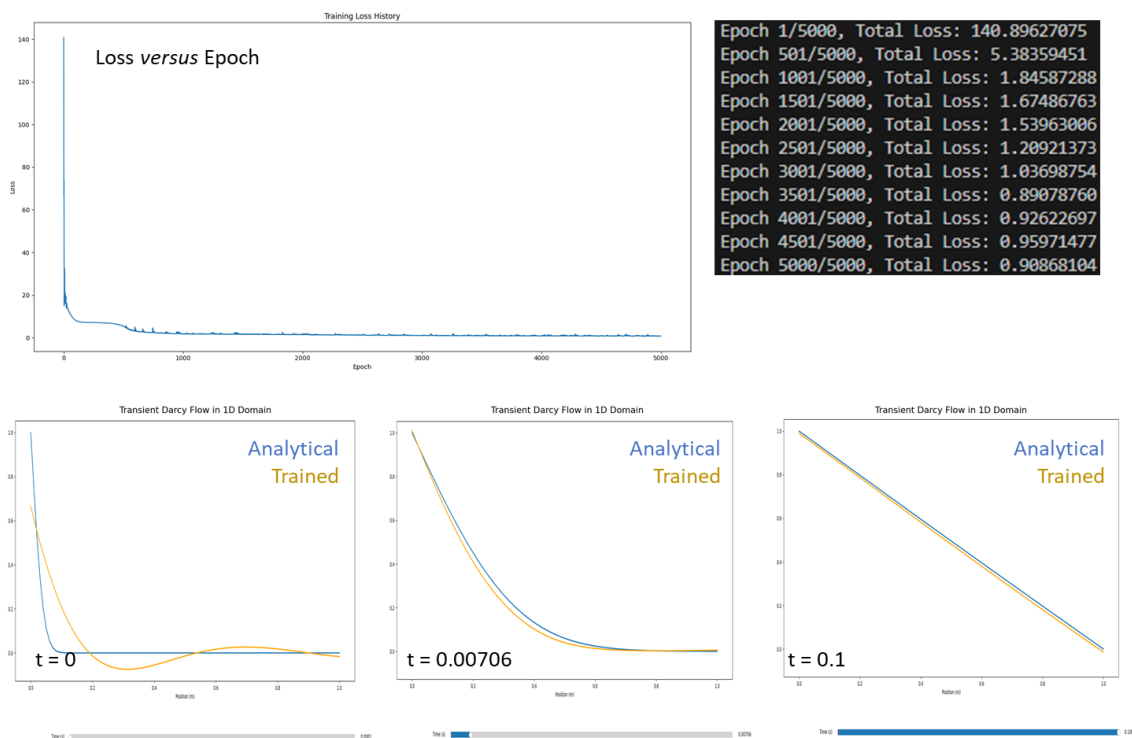


Figura 18: Resultados da ANN com camadas ocultas do caso 1.

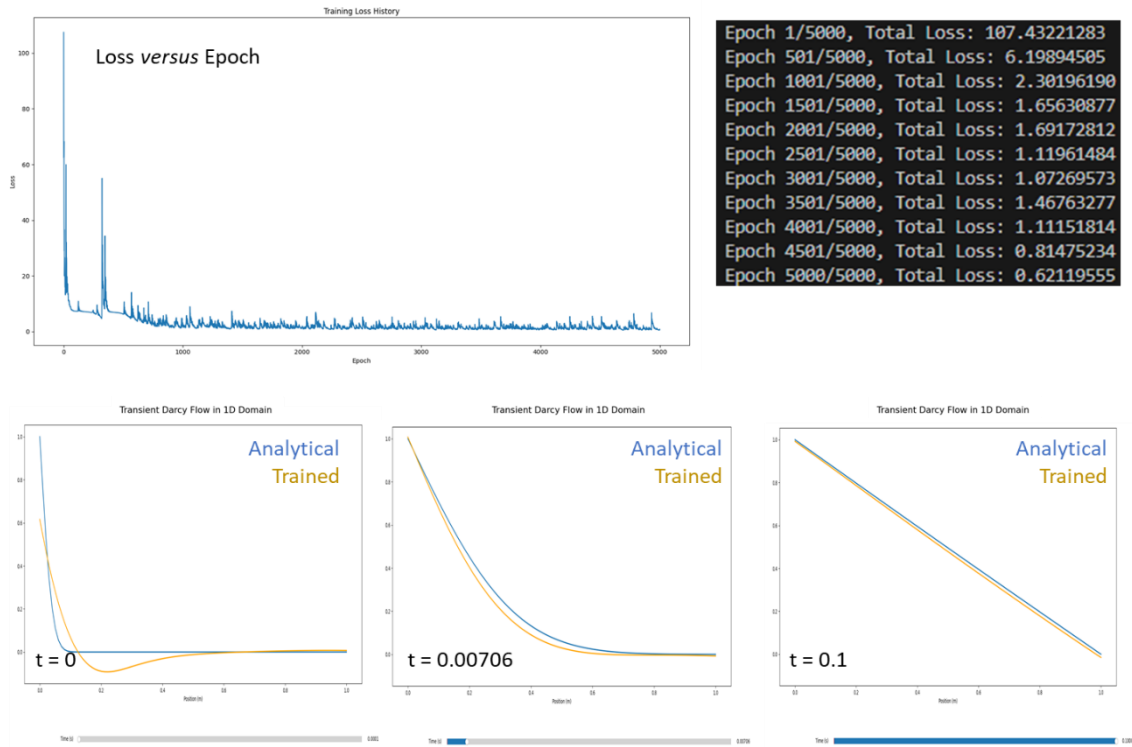


Figura 19: Resultados da ANN com camadas ocultas do caso 2.

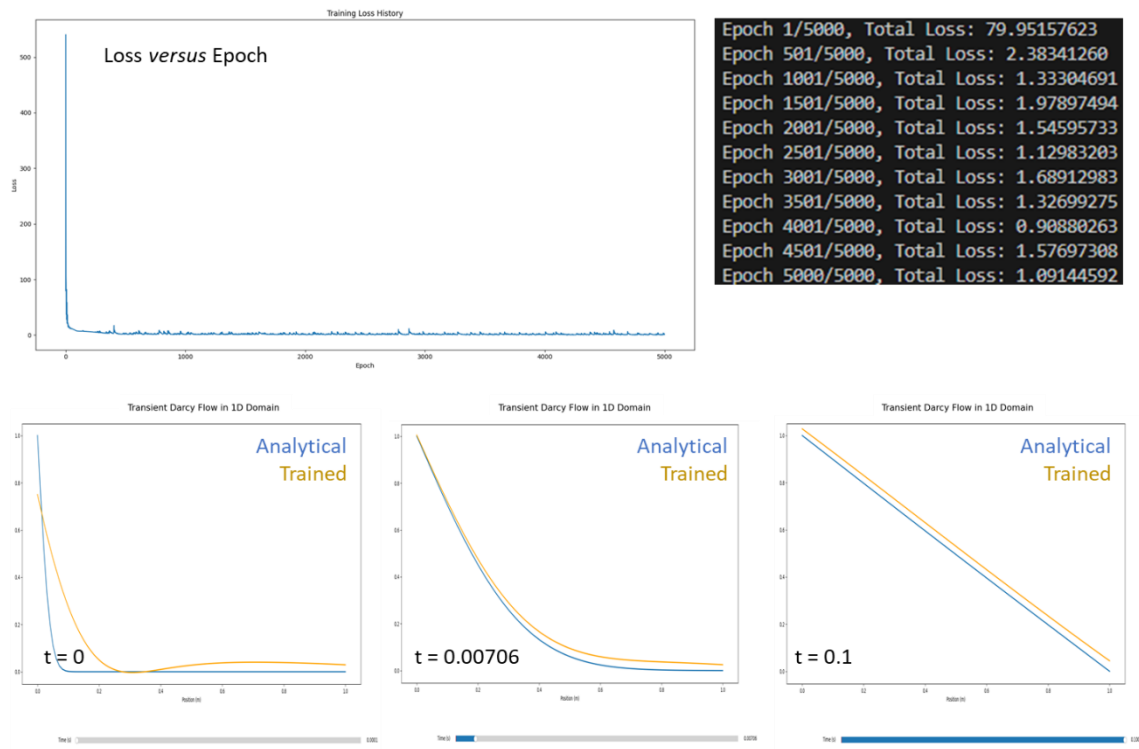


Figura 20: Resultados da ANN com camadas ocultas do caso 3.

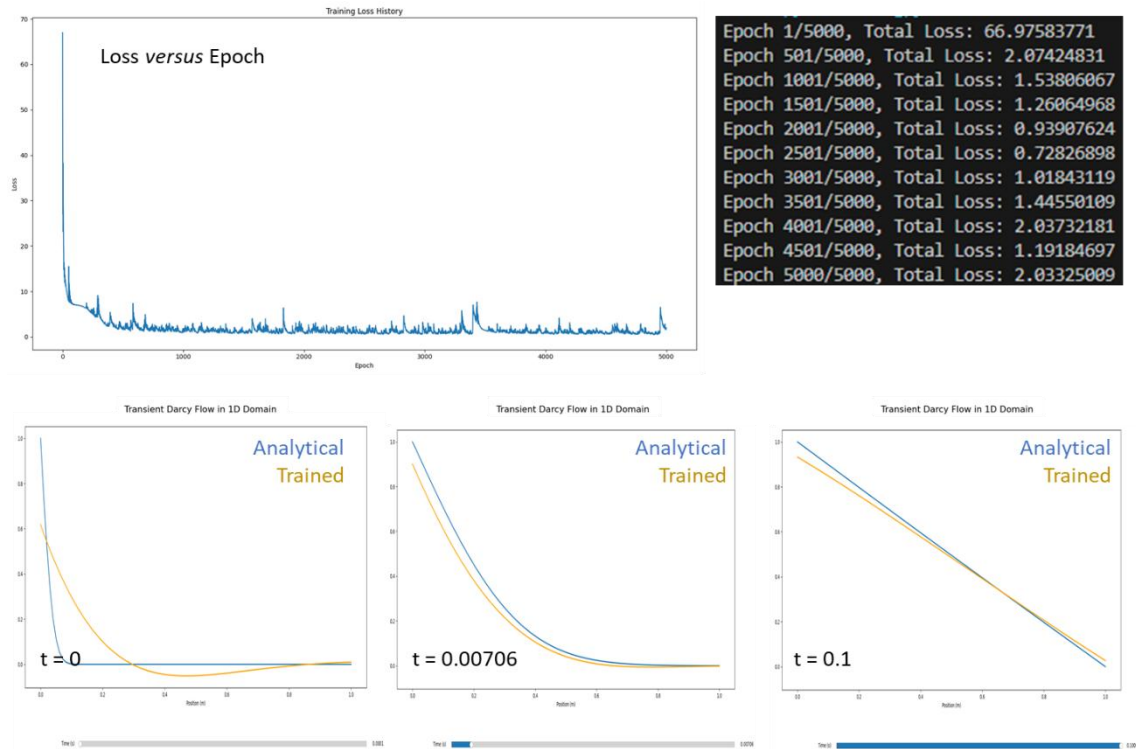


Figura 21: Resultados da ANN com camadas ocultas do caso 4.

Variando o tipo de função de ativação

Para testar a influência da função de ativação no problema estudado, foram realizados treinamentos da rede utilizando as seguintes funções de ativação: ReLU (Figura 22), LeakyReLU (Figura 23) e sigmoide (Figura 24).

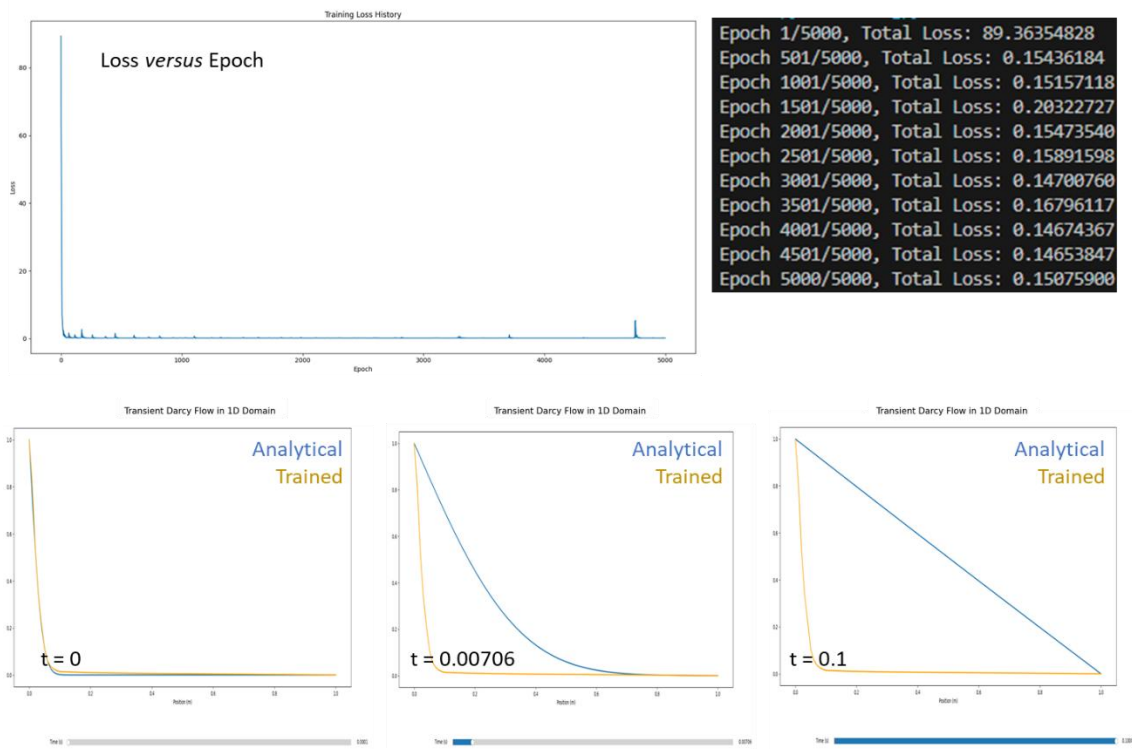


Figura 22: Resultados da ANN com ReLU.

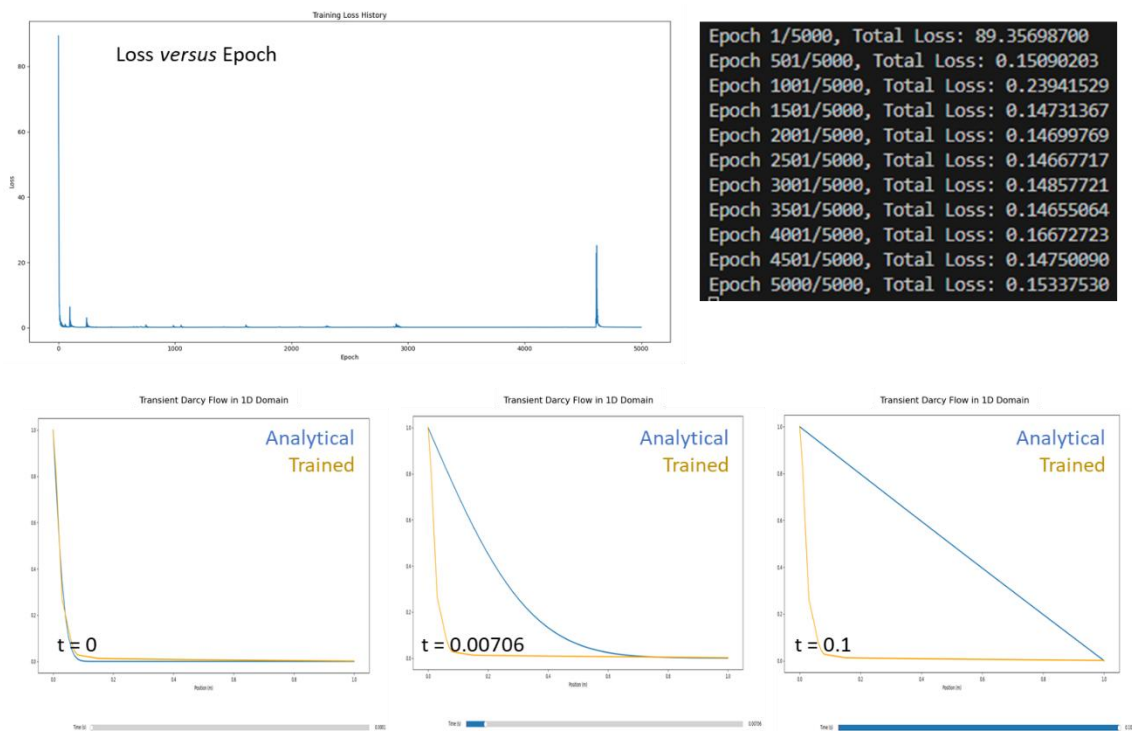
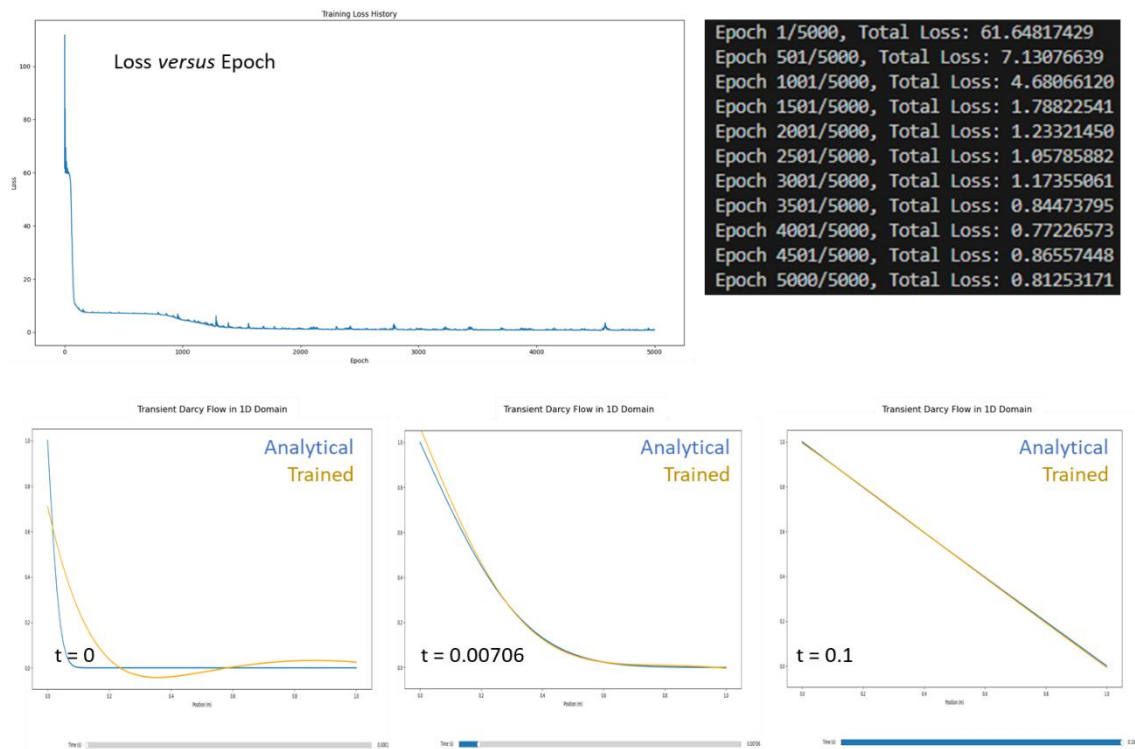


Figura 23: Resultados da ANN com LeakyReLU.



Resultados e discussões

Diante dos experimentos realizados, foi possível observar que o tamanho da amostra e os pesos utilizados nas funções de custo possuem um papel importante no treinamento da rede. Além disso, a utilização de redes maiores ou com mais neurônios não foi justificada com os resultados observados e melhores resultados foram obtidos utilizando as funções de ativação TanH e Sigmoide. Deste modo, a fim de estabelecer um bom caso de treino para posterior avaliação e estudo da estimativa de parâmetros, foi utilizada uma rede neural com essas considerações, sendo o número de amostras, hiperparâmetros e pesos das funções custo mostradas na Tabela 3.

Tabela 3: Dados para estudo da estimativa de parâmetros com a ANN

Número de amostras	10000
Fator de ruído	0.1
Taxa de aprendizado	0.01
Épocas	5000
Camadas ocultas	[50, 50, 50, 50]
Peso – condições de contorno	1000

Peso – equação diferencial	1
Peso – condição inicial	50
Função de ativação	Tangente Hiperbólica

Os resultados obtidos com o treinamento do caso apresentado na Tabela 3, são mostrados na Figura 25, onde pode-se notar um bom ajuste em tempos maiores de simulação e um ajuste razoável na condição inicial. Este caso foi utilizado para estimativa do parâmetro de porosidade.

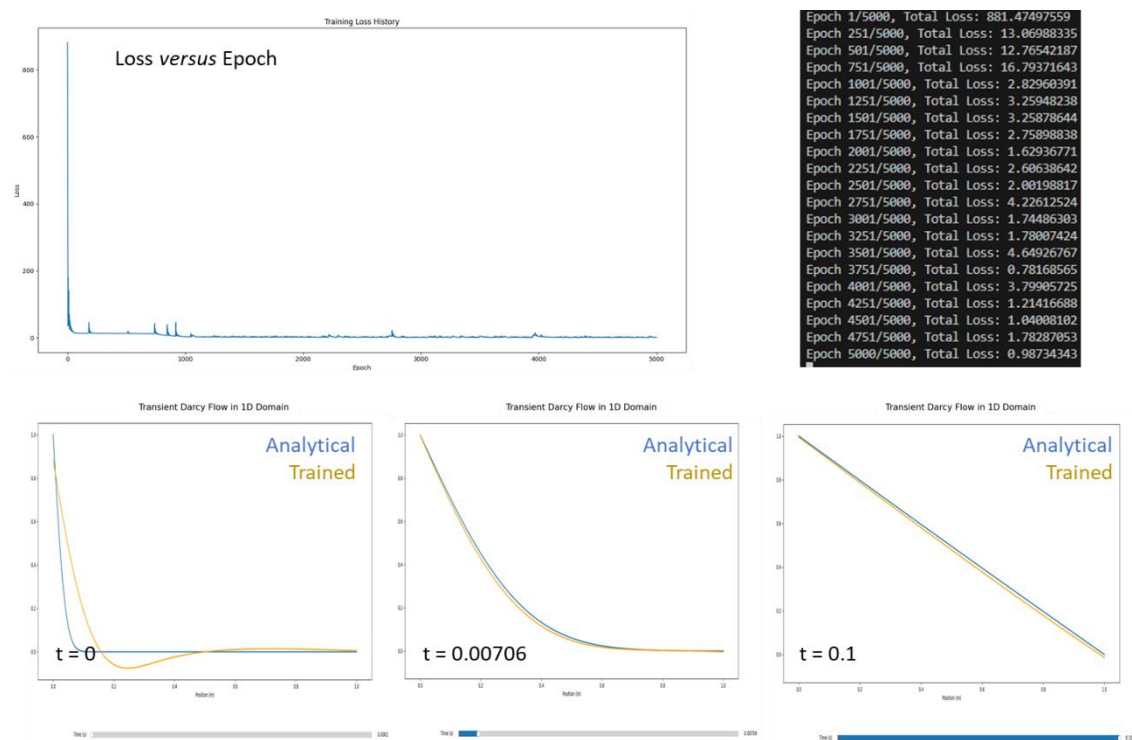


Figura 25: Resultados da ANN com o melhor ajuste obtido.

A estimativa da porosidade foi realizada para a configuração base mostrada na Tabela 3, sendo mostrado na Figura 26. A curva analítica mostrada se refere a equação analítica considerando o parâmetro estimado. Pode-se notar um bom ajuste entre a curva analítica e a curva treinada, contudo, o valor estimado para a porosidade não é similar ao utilizado para geração dos pontos de treinamento. O valor de porosidade estimado com o treinamento foi 4.656.

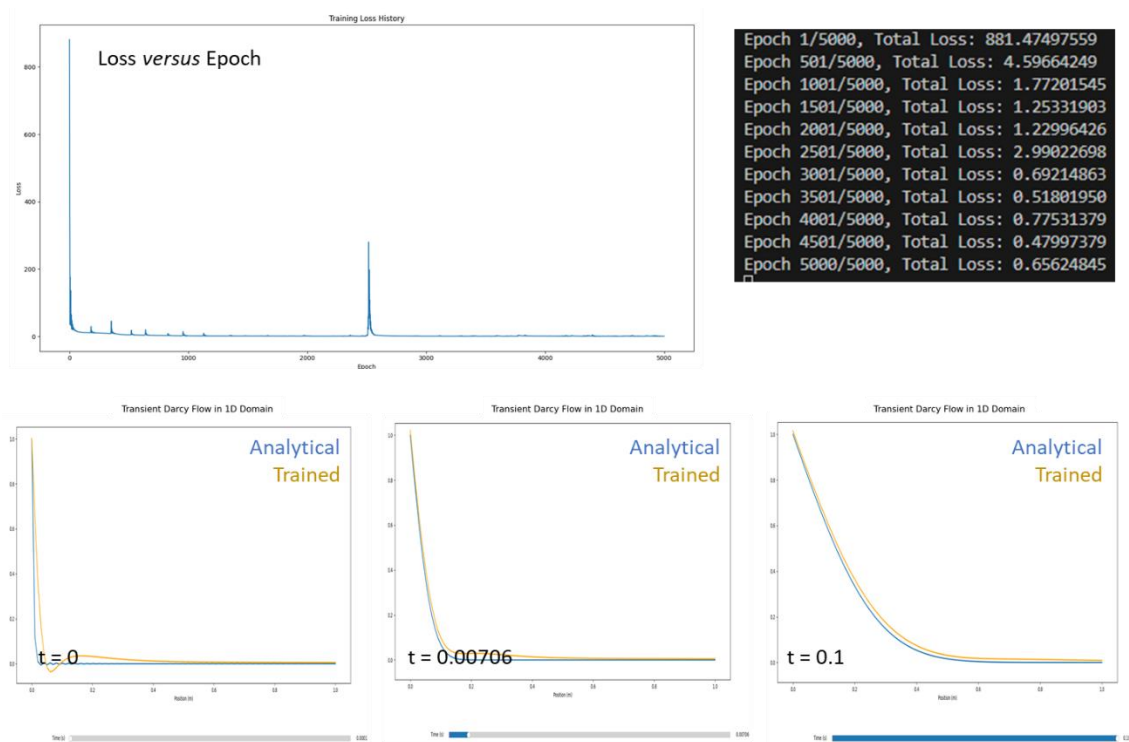


Figura 26: Resultados da ANN utilizando o parameter discovery.

Nota-se que a porosidade estimada destoa muito da utilizada nos pontos de treinamento, o que pode estar diretamente relacionado aos valores de peso utilizados para os custos da função diferencial. Deste modo, foram realizados treinamentos para diferentes valores de peso. A Figura 27 mostra essa comparação por meio da curva de razão entre a porosidade estimada e a utilizada para geração dos pontos de treinamento *versus* os valores de peso, onde pode-se notar que quanto maior o peso da função custo da equação diferencial, menor a razão entre as porosidades. Entretanto, a razão entre as porosidades tende a uma possível estabilidade longe do valor desejado.

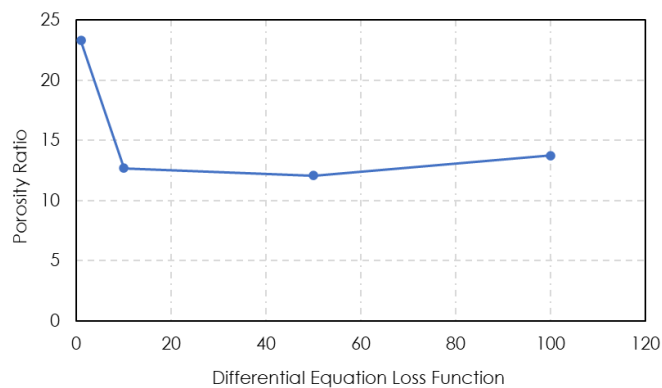


Figura 27: Comparação entre a porosidade estimada e utilizada para diferentes valores de peso da função diferencial – peso da condição inicial fixa em 50.

Deste modo, decidiu-se por reduzir o peso da função custo da condição inicial. Uma curva similar, na qual o peso da função custo da equação diferencial é variado, é mostrada na Figura 28, mas dessa vez com o peso da condição inicial reduzido para 0.01. Nota-se o mesmo comportamento anterior, mas com estabilidade observada para menor valor de razão entre porosidades, o que mostra que para uma melhor estimativa de coeficientes, a maior penalização da função custo da equação diferencial pode ser de grande importância.

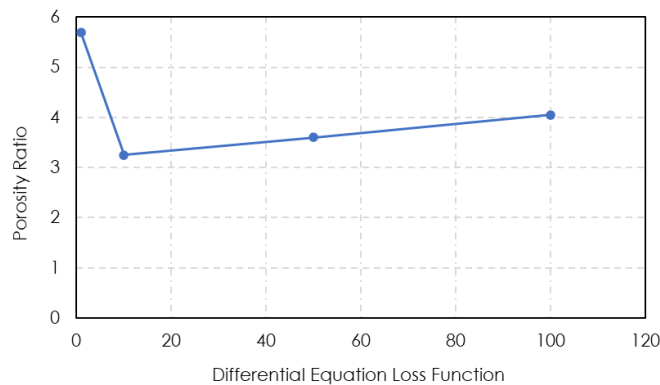


Figura 28: Comparação entre a porosidade estimada e utilizada para diferentes valores de peso da função diferencial – peso da condição inicial fixa em 0.01.

Conclusões

Para a maioria das diferentes configurações treinadas para a equação diferencial foi observada uma grande dificuldade da rede em ajustar a condição inicial do problema, mas que se torna menos evidente para maiores tempos de simulação. Em relação as configurações testadas:

- Aumentar o número de amostras se mostra positiva para o treinamento da rede, tendo em vista que melhores ajustes entre as curvas analítica e treinada são obtidos em maiores tempos de simulação.
- Dentre os níveis de ruídos testados neste trabalho, observa-se maior influência em estágios finais de simulação (quando o *steady-state* é alcançado), onde para o maior ruído treinado é observado um deslocamento da curva treinada em relação a analítica. Esse comportamento pode ser considerado esperado, visto que a inclusão de ruído levam o ajuste da reta para pontos distantes do valor analítico.
- Os pesos das funções de perda apareceram como um importante parâmetro. A utilização de valores com maiores proximidades entre os pesos dos custos das

condições de contorno, condição inicial e equação diferencial, apresentaram menor ajuste a curva analítica em tempos maiores.

- A utilização de redes mais largas ou com mais neurônios não se mostrou mais eficiente para ajuste da função treinada em comparação a função analítica. Além disso, a utilização da rede em formato triangular, muitas vezes apontado como uma boa solução, não se mostrou eficiente, apresentando um ajuste inferior as redes retangulares testadas.
- As funções de ativação ReLU e LeakyReLU não foram capazes de realizar o treinamento em função do tempo, apesar do melhor ajuste a condição inicial. Já as funções TanH e Sigmoide, apesar não realizarem um bom ajuste a condição inicial, foram capazes de realizar o treinamento do comportamento transiente.
- O ajuste do coeficiente (parameter discovery) se mostrou não trivial, tendo em vista que, por exemplo, os parâmetros de peso não foram similares aos utilizados para treinamento da rede, necessitando de ajustes pontuais para um bom ajuste entre as porosidades estimada e utilizada. Além disso, verificou-se que um melhor ajuste entre as porosidades foi obtido quando o peso da condição inicial foi muito reduzido e o peso da equação diferencial foi aumentada.
- Para trabalhos futuros destaca-se a necessidade de testes com maior quantidade de amostras, o que exige maior custo computacional, com o objetivo de obter um melhor ajuste entre a porosidade estimada e esperada. Além disso, pode ser benéfico para essa estimativa, realizar o treinamento sem os tempos iniciais, onde maior não linearidade é observada.

Referências

- [1] PIRES, Paulo Botelho; SANTOS, José Duarte; PEREIRA, Inês Veiga. Artificial Neural Networks: History and State of the Art. In: KHOSROW-POUR, Mehdi (Org.). Encyclopedia of Information Science and Technology, Sixth Edition. Hershey (PA): IGI Global, 2025. p. 1–25. doi: 10.4018/978-1-6684-7366-5.ch037.
- [2] SCHMIDHUBER, Jürgen. Deep learning in neural networks: An overview. Neural Networks, v. 61, p. 85–117, 2015. doi:10.1016/j.neunet.2014.09.003.
- [3] KINSLEY, Harrison; KUKIEŁA, Daniel. Neural Networks from Scratch in Python. 1. ed. [S.l.]: [s.n.], 2020.
- [4] GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. Deep Learning. Cambridge (MA): MIT Press, 2016. ISBN 978-0262035613.
- [5] RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, v. 378, p. 686–707, 2019. doi:10.1016/j.jcp.2018.10.045
- [6] Notas e aula do professor Jon Kleppe (jon.kleppe@ntnu.no). DERIVATION OF FLUID FLOW EQUATIONS. TPG4150 Reservoir Recovery Techniques 2017. Acesso em 01/12/2025: <<https://www.ipt.ntnu.no/~kleppe/TPG4150/equations.pdf>>